

Technical Debt & Risk Assessment

Project: SafeSpace
Assessment Date: February 5, 2026
Assessment Tool: Cursor AI Analysis

Executive Summary

This document identifies 5 significant areas of technical debt discovered during a comprehensive code review of the SafeSpace codebase. Items are classified under Architectural Debt, Test Debt, or Documentation Debt categories.

#	Item Name	Category	Severity
1	Client-Side Security Logic	Architectural Debt	Critical
2	Async State Management Issues	Architectural Debt	Critical
3	No Automated Test Suite	Test Debt	Critical
4	Hardcoded Business Logic	Architectural Debt	Medium
5	Missing Security Documentation	Documentation Debt	High

Part 1: Technical Debt Audit

Item 1: Client-Side Security Logic

Category: Architectural Debt

Description:

Security-critical logic exists only on the client side, creating multiple vulnerabilities. Content moderation in `src/lib/contentModeration.ts` can be bypassed by modifying client code or calling APIs directly. Multiple pages render user-generated content without sanitization, creating XSS attack vectors.

Affected Files:

- `src/lib/contentModeration.ts` (Lines 38-51) - Client-side filtering
- `src/pages/Journal.tsx` (Lines 253, 256) - Unsanitized content display
- `src/pages/Community.tsx` (Lines 362, 369, 432) - Unsanitized posts
- `src/components/MessageThread.tsx` (Lines 84-93) - No input validation

Code Example:

```
// contentModeration.ts - Client-side only, easily bypassed
export const censorContent = (content: string): string => {
  let censoredContent = content;
```

```
patterns.forEach((pattern) => {
  censoredContent = censoredContent.replace(pattern, (match) => {
    return '*'.repeat(match.length);
  });
});
return censoredContent;
};

// Journal.tsx - User content displayed without sanitization
<p className="text-sm whitespace-pre-wrap">{entry.content}</p>
```

Remediation Plan:

Refactor content moderation into a server-side service using Supabase Edge Functions or database triggers. Create a modular sanitization utility using DOMPurify for all user-generated content. Add Content Security Policy headers.

Item 2: Async State Management Issues

Category: Architectural Debt

Description:

Multiple hooks have race conditions and memory leaks from uncontrolled async operations. The authentication hook has a race between `onAuthStateChange` and `getSession()`. Other hooks perform async operations without cleanup, causing state updates after component unmount.

Affected Files:

- `src/hooks/useAuth.tsx` (Lines 21-39) - Auth state race condition
- `src/hooks/useUserStats.tsx` (Lines 50-90) - No cancellation for async ops
- `src/hooks/useDiscreetMode.tsx` (Lines 15-34) - State updates after unmount
- `src/components/MessageThread.tsx` (Lines 30-53) - No AbortController

Code Example:

```
// useAuth.tsx - Race condition between listener and getSession
useEffect(() => {
  const { data: { subscription } } = supabase.auth.onAuthStateChange(
    (event, session) => {
      setSession(session);
      setLoading(false); // Can fire before getSession resolves
    }
  );
});

// RACE CONDITION - both set loading state
supabase.auth.getSession().then(({ data: { session } }) => {
  setSession(session);
  setLoading(false);
});

return () => subscription.unsubscribe();
```

```
  }, []);

  // useUserStats.tsx - No cleanup, memory leak risk
  useEffect(() => {
    const loadStats = async () => {
      const { data } = await supabase.from('profiles').select('*');
      setStats(data); // Memory leak if component unmounted
    };
    if (user) loadStats(); // No cleanup returned
  }, [user]);
```

Remediation Plan:

Refactor auth hook to use a single source of truth for initialization. Implement AbortController pattern for all async operations. Create a reusable `useAsyncEffect` hook that handles cleanup automatically.

Item 3: No Automated Test Suite

Category: Test Debt

Description:

The project has zero test files. No unit tests, integration tests, or end-to-end tests exist. There is no testing framework configured, no mock utilities for Supabase, and no CI/CD pipeline for automated testing. AI-generated code has not been verified through "trust but verify" protocols.

Missing Infrastructure:

- No Vitest/Jest configuration
- No React Testing Library setup
- No Supabase mock utilities
- No test coverage reporting
- No pre-commit test hooks

Critical Untested Flows:

Feature	Risk Level
Authentication (sign up/in/out)	Critical
Content moderation logic	Critical
Journal CRUD operations	High
Friend request workflow	High
Message sending/receiving	High

Remediation Plan:

Install Vitest and React Testing Library. Create mock utilities for Supabase client. Write tests for critical authentication and content moderation flows first. Implement pre-commit hooks to require tests. Conduct security audit of AI-generated code, particularly regex patterns in content moderation.

Item 4: Hardcoded Business Logic and Fake Feature Data

Category: Architectural Debt

Description:

Business logic, scoring formulas, and configuration values are hardcoded throughout components rather than centralized. Magic numbers appear without explanation, and data that should come from the database is embedded in component files. Several features display fake/placeholder data that does not reflect actual user activity.

4.1 Hardcoded Feature Data (Should Be Database-Driven)

File	Line(s)	Hardcoded Data	Impact
Challenges.tsx	28-94	Entire challenges array (5 challenges)	Challenges cannot be managed without code changes
Calm.tsx	8-17	Affirmations array (8 items)	Users cannot customize calming content
Calm.tsx	50-56	Grounding steps (5-4-3-2-1 exercise)	Exercise cannot be modified
Calm.tsx	64-89	Exercises array (3 exercises)	New exercises require code deployment
Calm.tsx	32-35	Breathing durations (4000ms each)	Cannot adjust timing for users
Journal.tsx	36-42	Reflection prompts (5 prompts)	Limited variety, no personalization
Journal.tsx	30-34	Mood options (3 options)	Cannot add new mood categories
Profile.tsx	130-134	Resource URLs (3 resources)	Cannot update links without deployment

4.2 Fake/Placeholder Data (Non-Functional Features)

File	Line(s)	Fake Data	Issue
Community.tsx	38-44	Topic counts: 156, 43, 67, 28, 18	Static numbers, not calculated from actual posts
Community.tsx	82	Comments count always = 0	Comments feature not implemented
Profile.tsx	125-126	Achievements "Voice Found" and "Breath Master" always false	Achievement tracking not implemented
Profile.tsx	43	Notifications toggle state	Not persisted to database

File	Line(s)	Fake Data	Issue
Dashboard.tsx	21-25	"Today's Challenge" object	Same challenge every day, not dynamic

4.3 Magic Numbers (Undocumented Business Logic)

File	Line	Value	Purpose	Issue
Dashboard.tsx	20, 70	6	Weekly challenge goal	Why 6? Not documented
Profile.tsx	111-114	5, 2, 3, 100	Confidence score multipliers	Arbitrary, no rationale
Profile.tsx	367	"v1.0"	App version	Should come from package.json

4.4 Detailed Code Examples

Challenges.tsx - Entire feature is hardcoded (Lines 28-94):

```
// All 5 challenges are hardcoded – cannot be managed without code changes
const challenges: Challenge[] = [
  {
    id: "1",
    title: "Say hi to a classmate",
    description: "Greet someone in your class...",
    difficulty: "Easy",
    points: 10,
    category: "Social",
    tips: ["A simple smile and 'hi' is perfect", ...],
  },
  // ... 4 more hardcoded challenges
];
```

Community.tsx - Fake topic counts (Lines 38-44):

```
// These numbers are FAKE – they don't reflect actual post counts
const topics = [
  { name: "All", count: 156 }, // FAKE
  { name: "Wins", count: 43 }, // FAKE
  { name: "Support", count: 67 }, // FAKE
  { name: "College", count: 28 }, // FAKE
  { name: "Work", count: 18 }, // FAKE
];
```

Profile.tsx - Non-functional achievements (Lines 122-128):

```
const achievements = [
  { name: "First Step", earned: completedChallenges >= 1 }, // Works
  { name: "Week Warrior", earned: journalStreak >= 7 },      // Works
  { name: "Voice Found", earned: false },                    // ALWAYS
  FALSE - not tracked
  { name: "Breath Master", earned: false },                  // ALWAYS
  FALSE - not tracked
  { name: "Month Strong", earned: journalStreak >= 30 },     // Works
];
```

Remediation Plan**Phase 1: Database Schema**

- Create **challenges** table with all challenge data
- Create **calm_exercises** table for exercises and affirmations
- Create **reflection_prompts** table for journal prompts
- Add **calm_usage_count** column to profiles for "Breath Master" achievement

Phase 2: Dynamic Data Fetching

- Replace hardcoded arrays with Supabase queries
- Calculate topic counts from actual **community_posts** data
- Track calm tool usage for achievement system

Phase 3: Configuration

- Create **src/lib/constants.ts** for magic numbers
- Use **import.meta.env.VITE_APP_VERSION** for version string
- Document business logic for scoring formulas

Item 5: Missing Security Documentation

Category: Documentation Debt

Description:

Security-critical decisions are not documented. There is no explanation of why content moderation is client-side, what threats the app protects against, or how Row Level Security policies are designed. This makes security review and maintenance difficult.

Undocumented Security Areas:

- Content moderation approach and its limitations
- Authentication flow and session management decisions
- Row Level Security (RLS) policy design rationale
- Data privacy considerations for sensitive journal entries

- Friend request approval workflow security model

Remediation Plan:

Create `docs/security.md` documenting all security decisions and their rationale. Add inline comments explaining security-critical code sections. Document the threat model for the application. Create RLS policy documentation with test cases showing expected access control behavior.

Technical Debt Risk Matrix

Risk	Probability	Impact	Priority
XSS Attack via unsanitized content	Medium	Critical	P0
Content moderation bypass	High	High	P0
Auth state corruption	Medium	High	P1
Memory leaks in production	High	Medium	P1
Regression bugs (no tests)	High	High	P1

Part 2: AI & System Risk Assessment

Because SafeSpace was built using AI-augmented development (Lovable.dev), this risk assessment goes beyond standard software concerns to address risks specific to agentic systems.

Risk Category 1: Reliability & Hallucination

Risk 1.1: Hallucinated Regexp Patterns in Content Moderation

Description:

The content moderation system in `src/lib/contentModeration.ts` contains AI-generated regex patterns that have not been verified for correctness. The patterns may fail to catch harmful content, produce false positives on benign content, or contain ReDoS (Regular Expression Denial of Service) vulnerabilities.

Affected Code:

```
// src/lib/contentModeration.ts - AI-generated, unverified patterns
const patterns = sensitiveWords.map((word) => {
  const escapedWord = word.replace(/[-]/g, '[-\\s]?'); // May not handle
  all edge cases
  return new RegExp(`\\b${escapedWord}\\b`, 'gi');
});
```

Impact:

- Harmful content may slip through moderation, harming vulnerable users

- Benign mental health discussions may be incorrectly censored
- Malicious input could cause regex engine to hang (ReDoS)

Mitigation Strategy:
Require human verification of all regex patterns. Create a comprehensive test suite with edge cases. Use established content moderation libraries instead of custom regex. Implement regex timeout limits.

Risk Category 2: Security & Ethics

Risk 2.1: Sensitive Mental Health Data Exposure

Description:
SafeSpace stores highly sensitive mental health data (journal entries, mood tracking, social anxiety challenges) in Supabase. The AI-generated code lacks proper data classification, encryption-at-rest verification, and audit logging. Journal entries could be exposed through XSS attacks, API misconfigurations, or insider threats.

Affected Data:

Table	Sensitivity	Current Protection
journal_entries	Critical (mental health)	RLS only
community_posts	High (personal struggles)	RLS only
messages	High (private conversations)	RLS only
profiles	Medium (identity)	RLS only

Impact:

- Exposure of mental health data could cause severe harm to users
- Potential HIPAA/GDPR violations if data is considered health information
- Reputational damage and loss of user trust

Mitigation Strategy:
Implement field-level encryption for journal entries. Add comprehensive audit logging for all data access. Conduct privacy impact assessment. Review RLS policies with security expert. Implement data retention and deletion policies.

Risk Category 3: Dependency Risk

Risk 3.1: Lovable.dev Platform Lock-in

Description:
The codebase was generated by Lovable.dev and contained platform-specific dependencies (`lovable-tagger`) and patterns. While the direct dependency has been removed, the code structure, component patterns, and architectural decisions reflect Lovable.dev's opinionated approach, which may not align with long-term maintenance needs.

Remaining Lovable.dev Patterns:

- Component structure matches Lovable.dev templates
- Supabase integration follows Lovable.dev conventions
- File organization reflects Lovable.dev scaffolding
- Type definitions auto-generated by Lovable.dev

Impact:

- Difficulty integrating with other tools or frameworks
- Technical decisions made by AI may not be optimal
- Future Lovable.dev changes could affect similar patterns

Mitigation Strategy:

Document which patterns originated from Lovable.dev. Gradually refactor toward standard React/Vite conventions. Establish internal coding standards independent of any AI platform. Review and potentially regenerate Supabase types using official CLI.

AI Risk Matrix

Risk	Category	Probability	Impact	Priority
Sensitive data exposure	Security & Ethics	Medium	Critical	P0
Hallucinated regex patterns	Reliability	High	High	P1
Lovable.dev pattern lock-in	Dependency	Low	Medium	P3

Part 3: Backlog Integration Plan

The following GitHub issues have been created from the top 3 critical technical debt items.

GitHub Issue #1: Implement Server-Side Content Moderation and Input Sanitization

Labels: security, critical, architectural-debt

Priority: P0

Estimate: 5 story points

Description

Security-critical logic currently exists only on the client side, creating XSS vulnerabilities and allowing content moderation bypass. This issue addresses moving content moderation to the server and adding input sanitization across all user-generated content.

User Story

As a **user**, I want my data to be protected from malicious content injection, so that **I can safely use the app without risk of XSS attacks or exposure to harmful content.**

Tasks

- ☐ **Task 1.1:** Install DOMPurify dependency

```
npm install dompurify @types/dompurify
```

- ☐ **Task 1.2:** Create sanitization utility at `src/lib/sanitize.ts`

```
import DOMPurify from 'dompurify';

export const sanitizeHTML = (dirty: string): string => {
  return DOMPurify.sanitize(dirty, { ALLOWED_TAGS: [] });
};
```

- ☐ **Task 1.3:** Apply sanitization to all user content display points:
 - `src/pages/Journal.tsx` (lines 253, 256)
 - `src/pages/Community.tsx` (lines 362, 369, 432)
 - `src/components/MessageThread.tsx` (line 120+)
- ☐ **Task 1.4:** Create Supabase Edge Function for server-side content moderation
 - Create `supabase/functions/moderate-content/index.ts`
 - Move regex patterns from `src/lib/contentModeration.ts` to server
 - Add rate limiting and logging
- ☐ **Task 1.5:** Add database trigger for content validation on INSERT/UPDATE
 - Create migration for `community_posts` table trigger
 - Create migration for `messages` table trigger
- ☐ **Task 1.6:** Add Content Security Policy headers to `index.html`

Acceptance Criteria

- ☐ **AC1:** All user-generated content is sanitized before display using DOMPurify
- ☐ **AC2:** Content moderation runs server-side via Supabase Edge Function
- ☐ **AC3:** XSS payloads like `<script>alert('xss')</script>` are neutralized when displayed
- ☐ **AC4:** Content moderation cannot be bypassed by calling Supabase APIs directly
- ☐ **AC5:** CSP headers block inline script execution
- ☐ **AC6:** All existing tests pass (once test suite exists)
- ☐ **AC7:** Manual security test confirms XSS vectors are blocked

Definition of Done

- ☐ Code reviewed and approved
- ☐ All acceptance criteria verified

- ☐ No new linter errors introduced
- ☐ Documentation updated in [docs/security.md](#)

GitHub Issue #2: Fix Async State Race Conditions and Memory Leaks

Labels: [bug](#), [critical](#), [architectural-debt](#), [stability](#)

Priority: P1

Estimate: 3 story points

Description

Multiple React hooks have race conditions and memory leaks from uncontrolled async operations. The authentication hook races between [onAuthStateChange](#) and [getSession\(\)](#), and other hooks lack proper cleanup, causing state updates after component unmount.

User Story

As a **user**, I want the app to be stable and responsive, so that **I don't experience crashes, frozen screens, or inconsistent authentication states**.

Tasks

- ☐ **Task 2.1:** Refactor [src/hooks/useAuth.tsx](#) to fix auth state race condition

```
useEffect(() => {
  let isMounted = true;

  const { data: { subscription } } = supabase.auth.onAuthStateChange(
    (event, session) => {
      if (isMounted) {
        setSession(session);
        setUser(session?.user ?? null);
        setLoading(false);
      }
    }
  );

  // Remove redundant getSession call - listener handles initial state

  return () => {
    isMounted = false;
    subscription.unsubscribe();
  };
}, []);
```

- ☐ **Task 2.2:** Create reusable [useAsyncEffect](#) hook at [src/hooks/useAsyncEffect.ts](#)

```
export function useAsyncEffect(
  effect: (signal: AbortSignal) => Promise<void>,
  deps: DependencyList
) {
  useEffect(() => {
    const controller = new AbortController();
    effect(controller.signal);
    return () => controller.abort();
  }, deps);
}
```

- ☐ **Task 2.3:** Refactor `src/hooks/useUserStats.tsx` with AbortController pattern
- ☐ **Task 2.4:** Refactor `src/hooks/useDiscreetMode.tsx` with cleanup
- ☐ **Task 2.5:** Refactor `src/components/MessageThread.tsx` with AbortController
- ☐ **Task 2.6:** Add mounted state tracking to all async state updates

Acceptance Criteria

- ☐ **AC1:** `useAuth` hook initializes without race condition (single source of truth)
- ☐ **AC2:** No React warnings about "Can't perform state update on unmounted component"
- ☐ **AC3:** Rapidly navigating between pages does not cause state corruption
- ☐ **AC4:** Auth state is consistent after page refresh
- ☐ **AC5:** Memory profiler shows no leaked subscriptions after component unmount
- ☐ **AC6:** All async operations can be cancelled via AbortController

Definition of Done

- ☐ Code reviewed and approved
- ☐ All acceptance criteria verified
- ☐ Manual testing of rapid navigation completed
- ☐ No console warnings in development mode
- ☐ Browser memory profiler confirms no leaks

GitHub Issue #3: Establish Automated Testing Infrastructure

Labels: `testing`, `critical`, `test-debt`, `infrastructure`

Priority: P1

Estimate: 8 story points

Description

The project has zero test files. This issue establishes the testing infrastructure and creates baseline tests for critical flows including authentication and content moderation.

User Story

As a **developer**, I want automated tests for critical functionality, so that **I can make changes confidently without introducing regressions**.

Tasks

- ☐ **Task 3.1:** Install testing dependencies

```
npm install -D vitest @testing-library/react @testing-library/jest-dom
@testing-library/user-event jsdom msw
```

- ☐ **Task 3.2:** Create Vitest configuration at `vitest.config.ts`

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react-swc';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.ts'],
    globals: true,
    coverage: {
      reporter: ['text', 'html'],
      exclude: ['node_modules/', 'src/test/'],
    },
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});
```

- ☐ **Task 3.3:** Create test setup file at `src/test/setup.ts`

```
import '@testing-library/jest-dom';
import { afterEach } from 'vitest';
import { cleanup } from '@testing-library/react';

afterEach(() => {
  cleanup();
});
```

- ☐ **Task 3.4:** Create Supabase mock utilities at `src/test/mocks/supabase.ts`
- ☐ **Task 3.5:** Write authentication tests at `src/hooks/__tests__/useAuth.test.tsx`

- Test: User can sign up with email/password
 - Test: User can sign in with valid credentials
 - Test: User receives error for invalid credentials
 - Test: User can sign out
 - Test: Auth state persists across page refresh
- ☐ **Task 3.6:** Write content moderation tests at `src/lib/__tests__/contentModeration.test.ts`
 - Test: Sensitive words are censored correctly
 - Test: Benign content is not modified
 - Test: Crisis content is detected
 - Test: Edge cases (empty string, special characters)
 - Test: ReDoS protection (regex timeout)
- ☐ **Task 3.7:** Write component test at `src/components/__tests__/ProtectedRoute.test.tsx`
- ☐ **Task 3.8:** Add test scripts to `package.json`

```
{
  "scripts": {
    "test": "vitest",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest --coverage"
  }
}
```

- ☐ **Task 3.9:** Configure pre-commit hook for tests (optional)

Acceptance Criteria

- ☐ **AC1:** `npm test` runs successfully with Vitest
- ☐ **AC2:** At least 5 authentication tests pass
- ☐ **AC3:** At least 5 content moderation tests pass
- ☐ **AC4:** ProtectedRoute component has test coverage
- ☐ **AC5:** Test coverage report generates to `coverage/` directory
- ☐ **AC6:** Supabase client is properly mocked (no real API calls in tests)
- ☐ **AC7:** All tests complete in under 30 seconds

Definition of Done

- ☐ Code reviewed and approved
- ☐ All acceptance criteria verified
- ☐ CI/CD pipeline updated to run tests (if applicable)
- ☐ README updated with testing instructions
- ☐ Minimum 50% coverage on critical files (`useAuth.tsx`, `contentModeration.ts`)

Backlog Summary

Issue	Title	Priority	Estimate	Status
#1	Implement Server-Side Content Moderation and Input Sanitization	P0	5 pts	To Do
#2	Fix Async State Race Conditions and Memory Leaks	P1	3 pts	To Do
#3	Establish Automated Testing Infrastructure	P1	8 pts	To Do

Total Estimated Effort: 16 story points

Generated by Cursor AI Analysis - February 5, 2026